PADERBORN UNIVERSITY

Mini Project

Explainable Artificial Intelligence

Dr. Stefan Heindorf

Goal

Goal

Explain the predictions of a machine learning model on an RDF dataset.

Tasks

1. Pick an RDF dataset
2. Analyze the dataset
3. Train a machine learning model on the data
4. Evaluate the predictive performance of the model
5. Generate explanations of the model
6. Evaluate the explanations

You can be creative and work on your own idea!
(The suggestions below are only meant as inspiration; you may choose a different direction. If you are uncertain, talk to me.)

Organization

Group size

- max. 3 person

Submission via Panda

- Content:
  - Name, imt accounts, and matriculation number of team members
  - Link to a single GitHub/GitLab repository (grant access to heindorf)
- Deadline: July 6, 2026

Deliverables via GitHub/GitLab

- Code (commented and executable, only content on main branch is considered)
- Documentation (README.md)
- Dependencies (requirements.txt)
- Report (in pdf)

Report

Template

- LNCS: https://www.overleaf.com/latex/templates/springer-lecture-notes-in-computer-science/kzwwpvhwnvfj#.WuA4JS5uZpi
- Max. 10 pages (plus unlimited references)

Suggested structure

1. Abstract (~1 paragraph)
2. Introduction (~1 page)
3. Data analysis (~1 page)
4. Model Training & Evaluation (~1 page)
5. Model Explanation (~3 pages)
6. Conclusion (~1 paragraph)
7. Contributions of team members (~1 paragraph)
8. Optional: Acknowledgement (~ 1 paragraph)

Evaluation criteria

Decision: pass / no pass

Completeness

- Is the submission complete (code, documentation, dependencies, report)?
- Is the repository well-structured and the code is of high quality? (e.g., can easily be run, passes flake8 checks)

Technical quality

- Does the technical material make sense?
- Are the things tried reasonable?

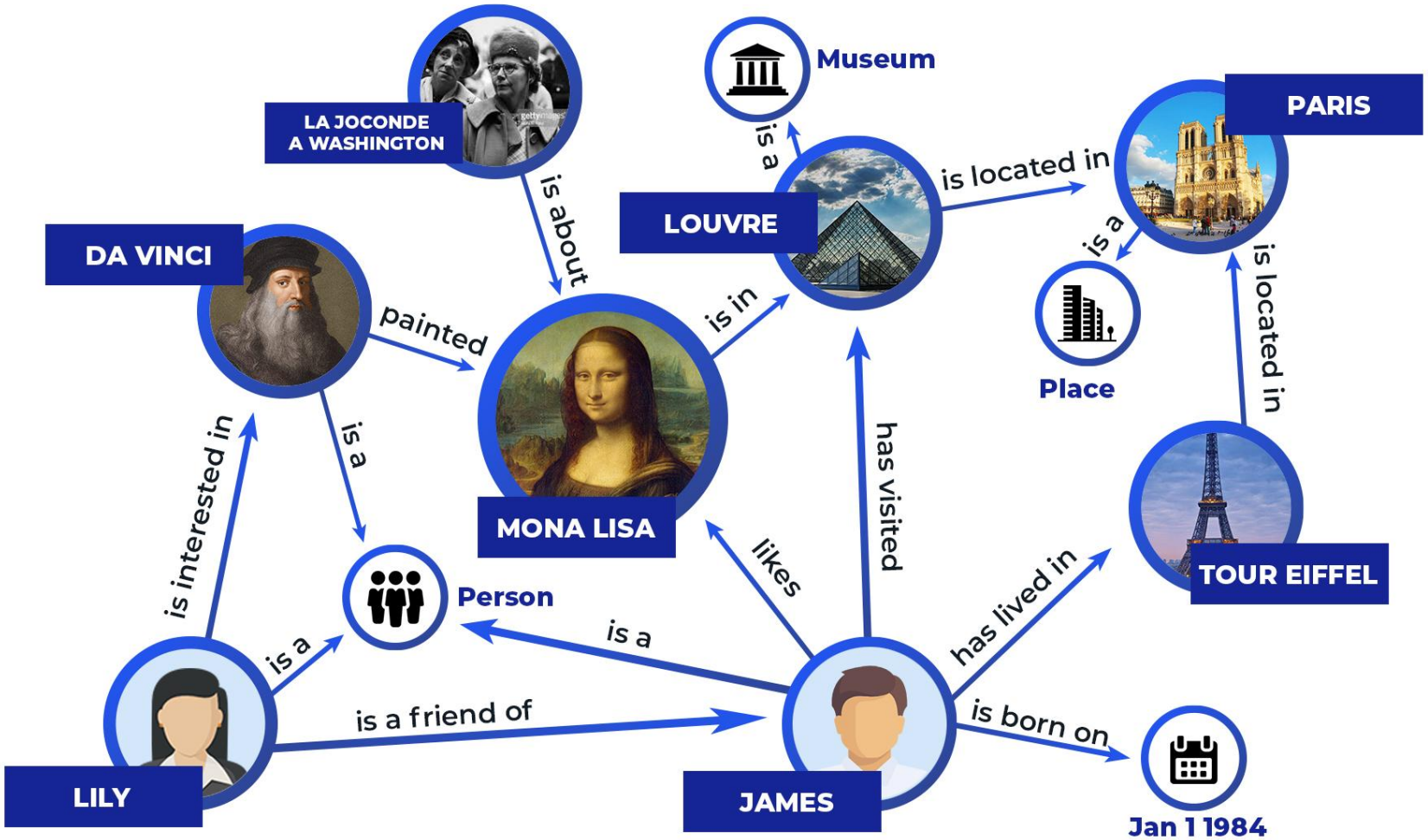
Originality

- Do the authors add their own data processing, methods, or analysis?
- Does the final project avoid being a pure mirror image of existing papers/projects?
  - It is not required to invent a new algorithm
  - Applying existing algorithms is sufficient
  - But: You cannot just copy an existing paper/project
  - You need to describe your approach in your own words and add your own ideas (e.g., choice of GNN algorithm, explanation method, examples you explain, …)

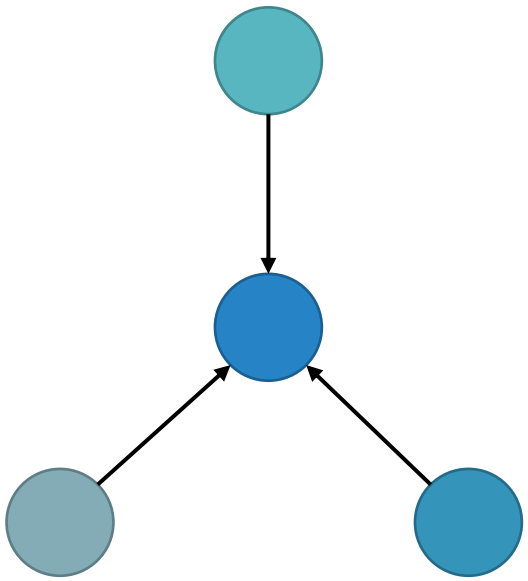
Communication

- Are the authors able to clearly and effectively explain the work that they did, including context, methods, and results?
- Does the report balance clarity with rigor?

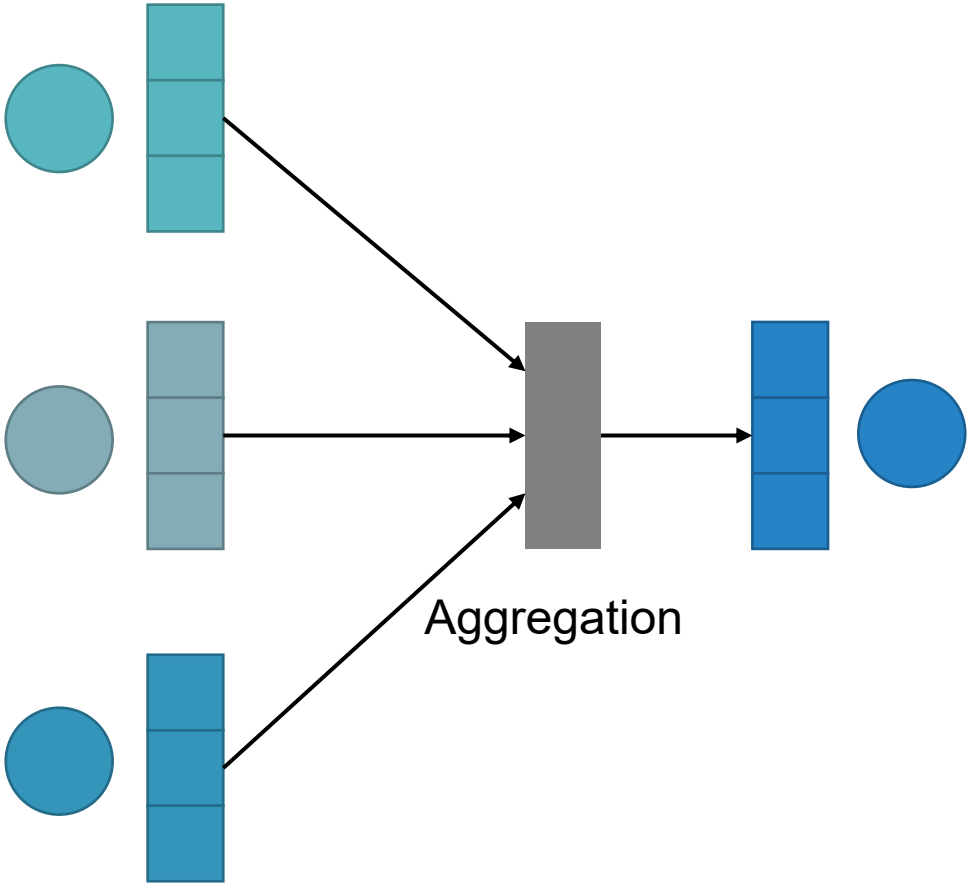
Graphs are everywhere
Knowledge graphs



Graph



Graph Neural Network



Datasets

Suggested RDF datasets

- AIFB: https://data.dgl.ai/dataset/rdf/aifb-hetero.zip
- MUTAG: https://data.dgl.ai/dataset/rdf/mutag-hetero.zip
- BGS: https://data.dgl.ai/dataset/rdf/bgs-hetero.zip
- AM: https://data.dgl.ai/dataset/rdf/am-hetero.zip

Other RDF datasets are possible

- Examples from KGBench (Amplus, Dmgfull, Dmg777k, DBLP, Mdgenre)
- Real-world datasets (DBpedia, Wikidata, YAGO, …)
- Talk to me if you are unsure

Tips

- Pick a small dataset (such that the computations can be done on your laptop in a small amount of time)
- Make yourself familiar with the dataset (e.g., compute some statistics, look at some examples, …)
- Start simple and improve your solution step by step
- Create a timeplan with deadlines, responsibilities, and a buffer at the end

Code, documentation & dependencies

Python code

- It should be easy to reproduce your results
- Clearly state which dataset was used and how it can be generated or downloaded
- Make sure that the (Python) version and all dependencies are specified
- Clearly state how the (Python) environment needs to be set up
- Clearly state which commands need to be run in which order to reproduce the results in your report (should be a small number of commands)

Documentation

- README file (with installation instructions, and steps to reproduce results)
- Comments in Code

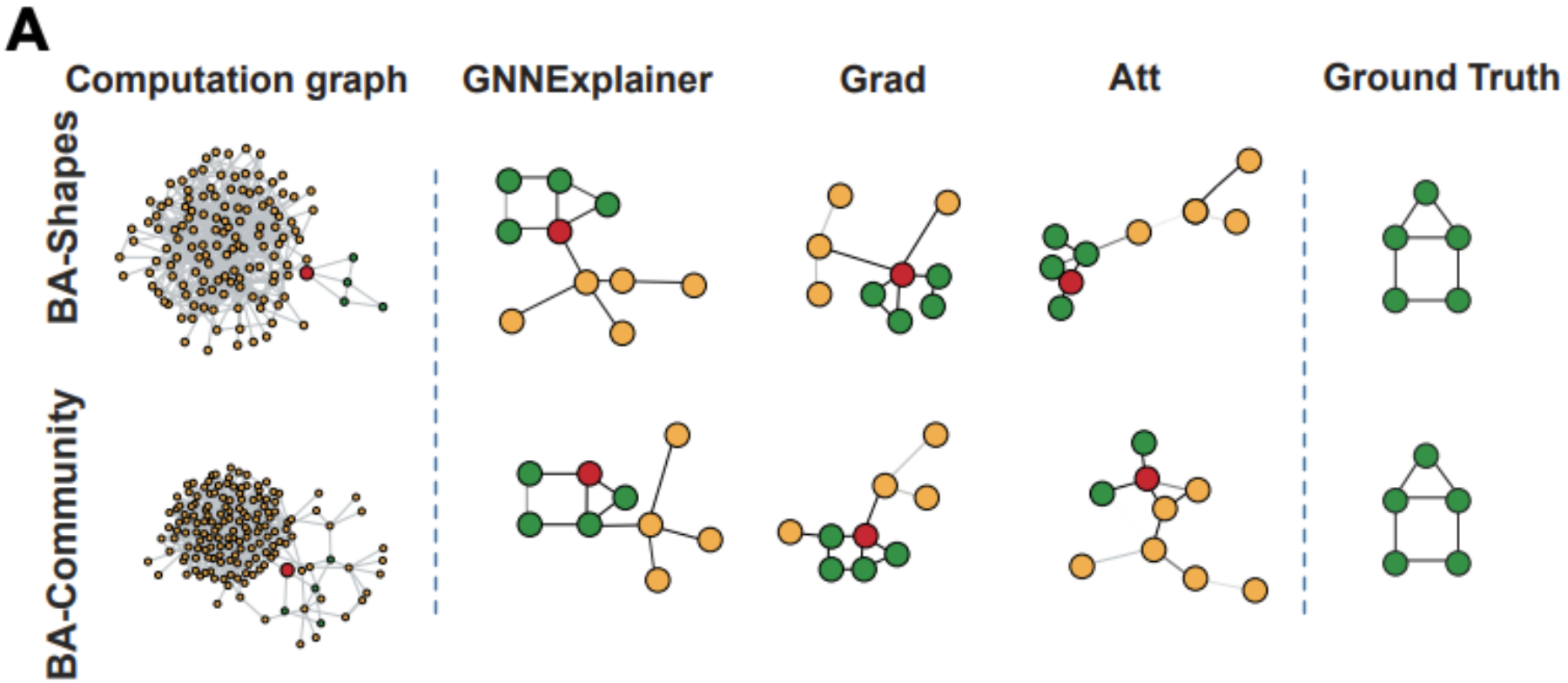
Software to train a graph neural network

- PyG: PyTorch Geometric (recommended): https://pytorch-geometric.readthedocs.io/en/latest/
- DGL: Deep Graph Library: https://docs.dgl.ai/

Explaining the predictions

Strategy 1: Explainability methods for GNNs

- PyTorch Geometric Explainer: https://pytorch-geometric.readthedocs.io/en/latest/modules/explain.html
- DGL's GNNExplainer: https://docs.dgl.ai/en/0.9.x/generated/dgl.nn.pytorch.explain.GNNExplainer.html



Source: https://arxiv.org/pdf/1903.03894.pdf

Explaining the predictions

Strategy 2: Explain ML model on tabular data

Strategy 2a: **Explain tabular ML model**

- Convert the graph data to tabular data
- Train a classical ML algorithm that operates on tabular data
- Explain the ML model

Strategy 2b: **Explain GNN via tabular surrogate model**

- Train a GNN (e.g., R-GCN) on the graph data
- Convert the graph data to tabular data and store a mapping (e.g., nodes to rows, edges to columns, …)
- Train a surrogate model that works on tabular data and approximates the graph neural network (surrogate model is trained on GNN predictions)
- Explain the surrogate model
  - If interpretable, explain it directly (rules, decision tree, linear model)
  - If not interpretable, use explainer (e.g., LIME, SHAP, …)

Explaining the predictions

Strategy 3: Global explanation of GNNs with description logics

There are a number of interpretable concept learners

- Inputs:
  - Knowledge base (in OWL format)
  - Set of positive and negative examples (nodes in a KG)
- Output:
  - Concept in description logics (such that most of the positive examples are instances of the concept and most of the negatives are not)

Concept learners

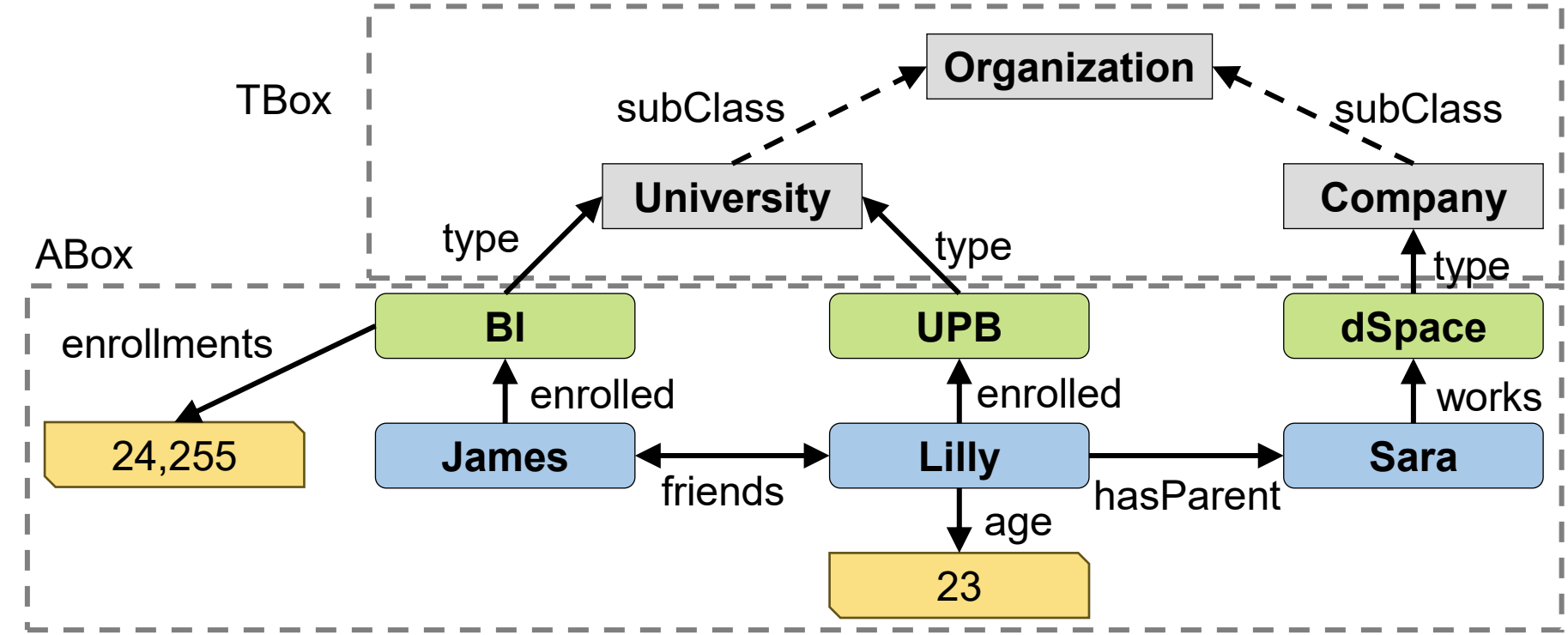
- Ontolearn: EvoLearner, CELOE (in Python, recommended) https://github.com/dice-group/Ontolearn
- DL-Learner (in Java): https://dl-learner.org/

Converting RDF to OWL

- Do it directly via RDFLib by iterating over the graph (recommended)
- Alternative: ROBOT library: https://robot.obolibrary.org/convert.html

Explaining the predictions

Strategy 3: Global explanation of GNNs with description logics



Training:
James: Student
Sara: **not** Student

Learned concept:
∃enrolled.University ≡ Student

Prediction: Is Lilly a student?

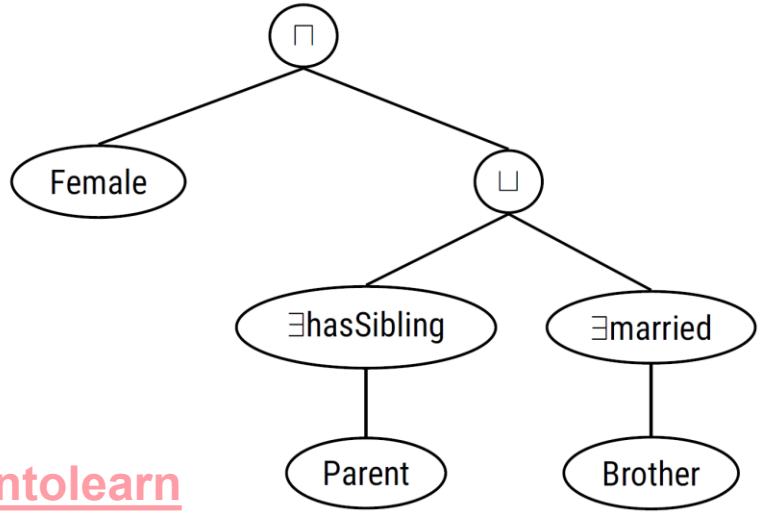
concept: yes, ∃enrolled.University

Explaining the predictions

Strategy 3: Global explanation of GNNs with description logics

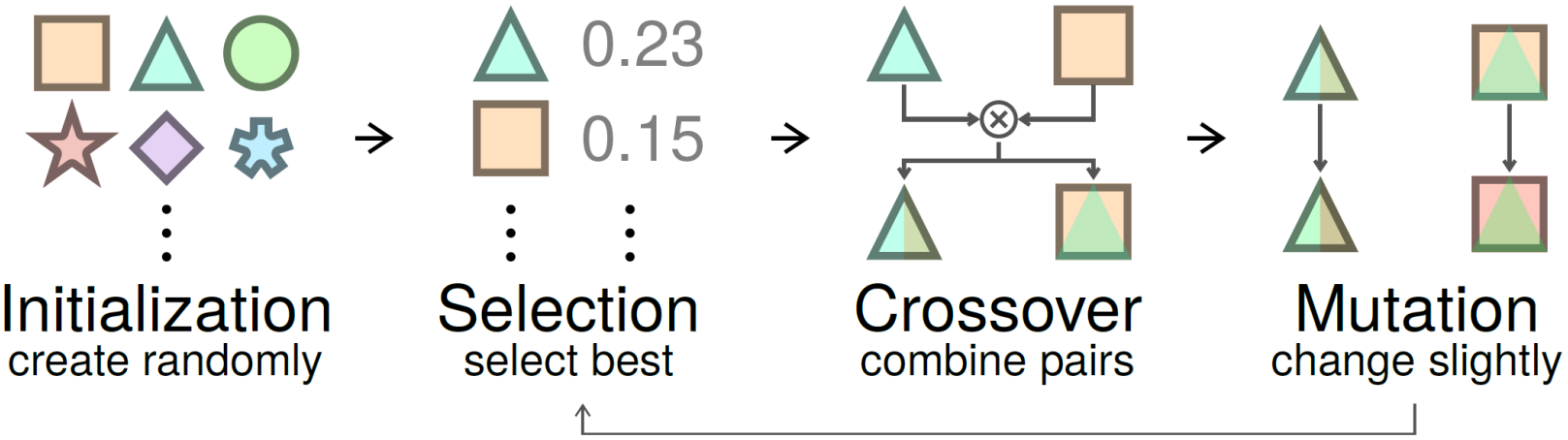
EvoLearner

- Learns description logic concepts (=**hypotheses**) on knowledge graphs
- Uses genetic programming: concepts are **represented as trees**
- **Paper:** https://arxiv.org/abs/2111.04879
- **Code:** https://github.com/dice-group/Ontolearn



- **Symbolic search**

Female ⊓ (∃hasSibling.Parent ⊔ ∃married.Brother)



Getting started

Some helpful tips to get started
(only suggestions, can be done differently)

Installation of Libraries

Installation of Pytorch Geometric (PyG)

Linux (highly recommended)

- Use native Linux machine
- Use virtual machine with Virtual Box
- Use Windows Subsystem for Linux

Google Colab

- https://colab.research.google.com/
- Easy setup
- GPU support can be enabled

Windows:

- It should also work with precompiled Python, PyTorch, PyG, … packages
- It is highly recommended to use Anaconda
- No guarantee that it works

Setup Python Environment

(tested on Ubuntu Linux 22.04 with GPU, might be adapted to your machine))

```
conda create -n xai26-mini python=3.12

conda activate xai26-mini

pip install torch torchvision torchaudio --index-url
https://download.pytorch.org/whl/cu126

pip install torch-geometric

pip install rdflib ipykernel pandas matplotlib scikit-learn

python -m ipykernel install --user --name xai26-mini --display-name
"Python (xai26-mini)"
```

Optional: check CUDA support

(requires GPU)

```
python
import torch
torch.cuda.is_available()

tensor = torch.zeros(5)
tensor = tensor.to("cuda")
tensor.is_cuda
```

Data Analysis

Dataset Questions

Before you start, brainstorm questions you would like to answer.

Examples:

- What are the node types in the dataset?
- What are the node features in the dataset?
- What are the edge types in the dataset?

- How many nodes are in the dataset?
- How many edges are in the dataset?

- What are the most frequent node types?
- What are the most frequent edge types?

- What are the most frequent patterns in the dataset?
  (consisting of 2 nodes, 3 nodes, …)

- And many more

AIFB Dataset

Overview

- Source: https://figshare.com/articles/dataset/AIFB_DataSet/745364/1
- Describes persons and their publications in different research groups
- Task: Given a person, predict which research group they belong to

Labels

URL	Research Group	Members
id1instance	Business Information and Communication Systems	73
id2instance	Efficient Algorithms	28
id3instance	Knowledge Management	60
id4instance	Complexity Management	16
id5instance	Usability Engineering	1

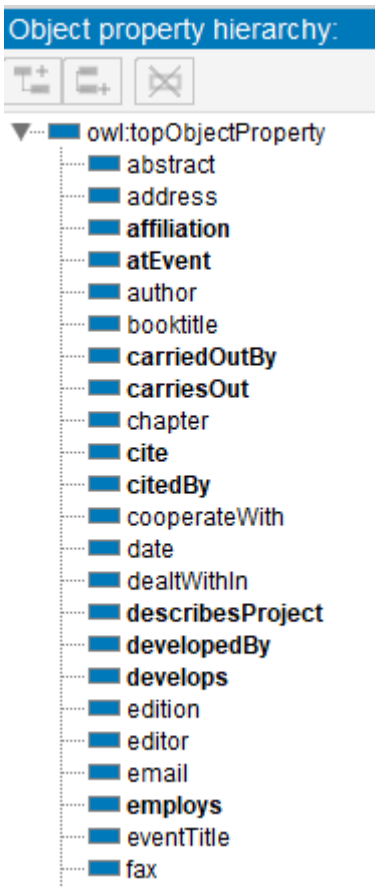
Explore Dataset in Text Editor

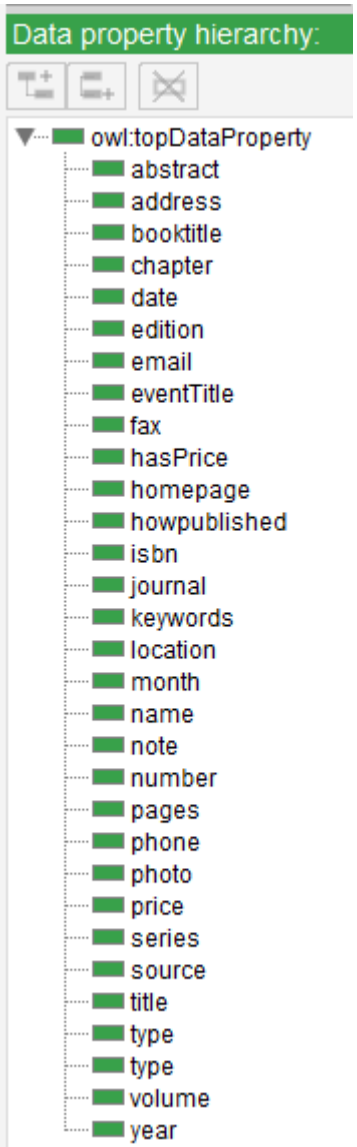
```
<http://www.aifb.uni-karlsruhe.de/Forschungsgruppen/viewForschungsgruppeOWL/id3instance>     a :ResearchGroup;
     :name "Knowledge Management"^^<http://www.w3.org/2001/XMLSchema#string>;
     :publishes <http://www.aifb.uni-karlsruhe.de/Publikationen/viewPublikationOWL/id0instance>,
           <http://www.aifb.uni-karlsruhe.de/Publikationen/viewPublikationOWL/id1001instance>,
           <http://www.aifb.uni-karlsruhe.de/Publikationen/viewPublikationOWL/id1003instance>,
           <http://www.aifb.uni-karlsruhe.de/Publikationen/viewPublikationOWL/id1004instance>,
           <http://www.aifb.uni-karlsruhe.de/Publikationen/viewPublikationOWL/id1005instance>,
```

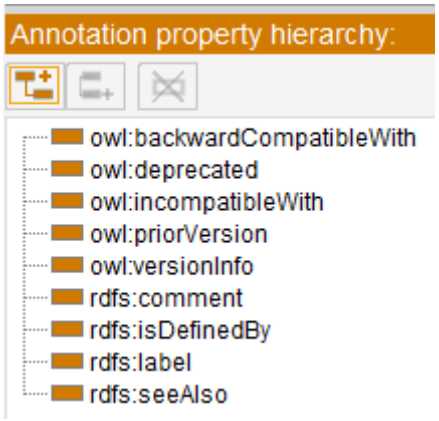
```
<http://www.aifb.uni-karlsruhe.de/Personen/viewPersonOWL/id11instance>     a :Person;
     :affiliation <http://www.aifb.uni-karlsruhe.de/Forschungsgruppen/viewForschungsgruppeOWL/id3instance>;
     :fax ""^^<http://www.w3.org/2001/XMLSchema#string>;
     :name "Stefan Decker"^^<http://www.w3.org/2001/XMLSchema#string>;
     :phone "-"^^<http://www.w3.org/2001/XMLSchema#string>;
     :publication <http://www.aifb.uni-karlsruhe.de/Publikationen/viewPublikationOWL/id1345instance>,
           <http://www.aifb.uni-karlsruhe.de/Publikationen/viewPublikationOWL/id143instance>,
           <http://www.aifb.uni-karlsruhe.de/Publikationen/viewPublikationOWL/id245instance>,
           <http://www.aifb.uni-karlsruhe.de/Publikationen/viewPublikationOWL/id248instance>,
           <http://www.aifb.uni-karlsruhe.de/Publikationen/viewPublikationOWL/id251instance>,
```

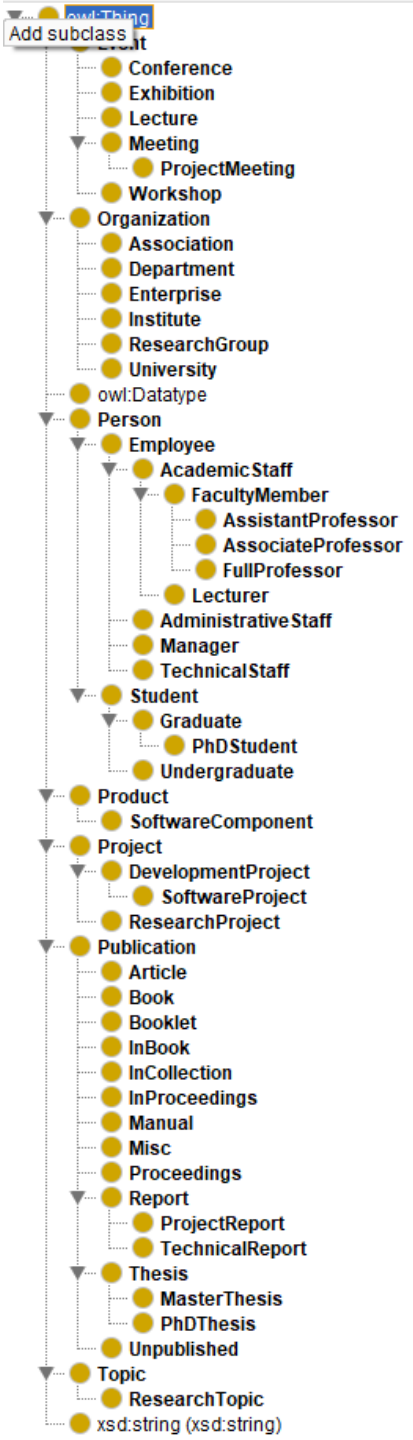
Explore Dataset in Protégé

https://protege.stanford.edu/

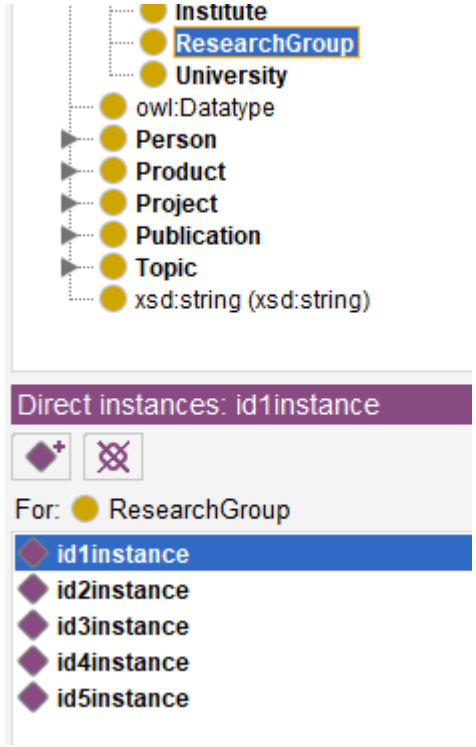








Explore Dataset in Protégé (II)



Explore Dataset in RDFLib

[28]:
```
import rdflib as rdf
from rdflib import Graph

from collections import Counter
```

[29]:
```
g = Graph()
g.parse("aifb-fixed_complete.n3")
```

[29]:
```
<Graph identifier=N1f7827b61570442b909838eeaffbbe74 (<class 'rdflib.graph.Graph'>)>
```

[30]:
```
print(len(g))
```

```
29226
```

[31]:
```
affiliation = rdf.term.URIRef("http://swrc.ontoware.org/ontology#affiliation")

src = []
dst = []

for s, p, o in g.triples((None, affiliation, None)):
    src.append(str(s))
    dst.append(str(o))

Counter(dst)
```

[31]:
```
Counter({'http://www.aifb.uni-karlsruhe.de/Forschungsgruppen/viewForschungsgruppeOWL/id1instance': 73,
         'http://www.aifb.uni-karlsruhe.de/Forschungsgruppen/viewForschungsgruppeOWL/id3instance': 60,
         'http://www.aifb.uni-karlsruhe.de/Forschungsgruppen/viewForschungsgruppeOWL/id2instance': 28,
         'http://www.aifb.uni-karlsruhe.de/Forschungsgruppen/viewForschungsgruppeOWL/id4instance': 16,
         'http://www.aifb.uni-karlsruhe.de/Forschungsgruppen/viewForschungsgruppeOWL/id5instance': 1})
```

[32]:
```
subject = rdf.term.URIRef('http://www.aifb.uni-karlsruhe.de/Forschungsgruppen/viewForschungsgruppeOWL/id3instance')
name = rdf.term.URIRef('http://swrc.ontoware.org/ontology#name')

for s, p, o in g.triples((subject, name, None)):
    print(f"{s}, {p}, {o}")
```

```
http://www.aifb.uni-karlsruhe.de/Forschungsgruppen/viewForschungsgruppeOWL/id3instance, http://swrc.ontoware.org/ontology#name, Knowledge Management
```

Strategy 1

Explainability methods for GNNs

Model Training

Suitable model architectures for RDF data
(data with many different relations)

- R-GCN
- R-GAT
- (Relational) Graph Transformers
- …

Evaluate your model both on the training and the test set to make sure that you achieve reasonable performance

Explanation approaches

- GNNExplainer
- PGExplainer
- SubGraphX
- …

One example to use R-GCN and GNNExplainer can be found in Panda
(it also contains additional ideas for potential mini project contributions)

Strategy 2

Explain ML model on tabular data

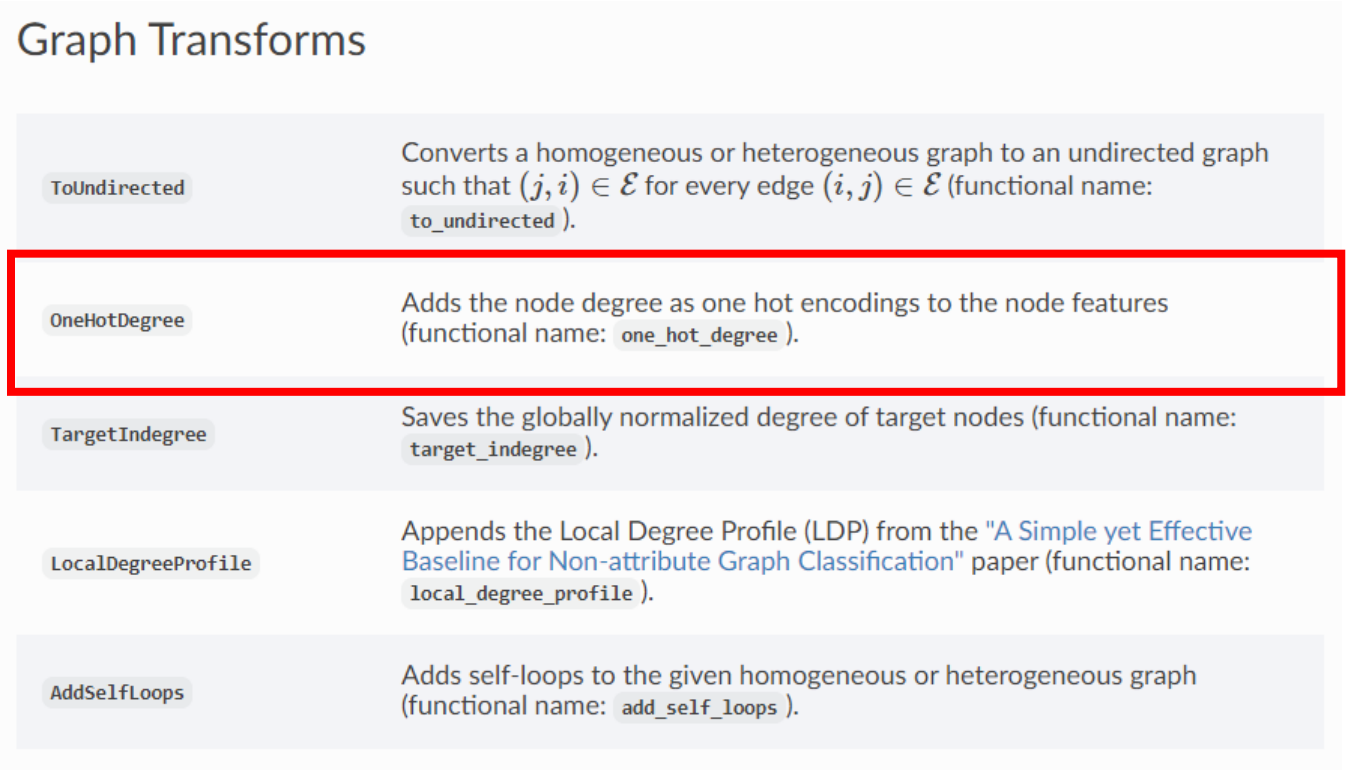
Converting RDF data to tabular data

- RDF library: https://rdflib.readthedocs.io/en/stable/
- "Table" library: https://pandas.pydata.org/docs/
- Given: set of triples: (subject, predicate, object)
- Each subject becomes a row in the table
- Each predicate becomes a column in the table
- Each object is inserted into a cell in the table
  - If there are multiple objects for one subject and predicate, a cell contains multiple values
  - If a subject does not have a certain predicate, the cell contains "NA" indicating a missing value
  - The best way to deal with multiple values and NA values depends on dataset and use case
- Dealing with multiple values and NA values
  - Simple approach: One-hot-encoding: for each combination of predicate and object, create a column with a binary feature (0 or 1) indicating whether the subject is connected to the object via the predicate
- It is okay to convert only part of a dataset to a table (but try to make sure to achieve an appropriate fidelity, i.e., the model trained on tabular data approximates the graph neural network well)

There are many ways how to convert RDF to a Dataframe

Finding a good conversion is part of feature engineering

Feature engineering for graphs



Graph Transforms

`ToUndirected`	Converts a homogeneous or heterogeneous graph to an undirected graph such that $(j, i) \in \mathcal{E}$ for every edge $(i, j) \in \mathcal{E}$ (functional name: `to_undirected`).
`OneHotDegree`	Adds the node degree as one hot encodings to the node features (functional name: `one_hot_degree`).
`TargetIndegree`	Saves the globally normalized degree of target nodes (functional name: `target_indegree`).
`LocalDegreeProfile`	Appends the Local Degree Profile (LDP) from the "A Simple yet Effective Baseline for Non-attribute Graph Classification" paper (functional name: `local_degree_profile`).
`AddSelfLoops`	Adds self-loops to the given homogeneous or heterogeneous graph (functional name: `add_self_loops`).

Perform random walks

(e.g., RDF2Vec, INK, path ranking algorithm [PRA], …)

ML and explanation algorithms for tables

Machine Learning Algorithms

- Scikit-Learn: https://scikit-learn.org/
- PyTorch: https://pytorch.org/

Interpretable white box models

- Rule learning: https://github.com/csinva/imodels
- Decision trees: https://scikit-learn.org/stable/modules/tree.html
- Linear models: https://scikit-learn.org/stable/modules/linear_model.html

Explanation methods for black box models

- LIME: https://github.com/marcotcr/lime
- SHAP: https://github.com/shap/shap
- (and many others mentioned throughout the lecture and exercises)

Explanation methods for neural networks

- Captum: https://captum.ai/

Strategy 3

Global explanation of GNNs with description logics

Starting point: EDGE paper

https://dl.acm.org/doi/10.1145/3627673.3679904

EDGE: Evaluation Framework for Logical vs. Subgraph Explanations for Node Classifiers on Knowledge Graphs

Rupesh Sapkota
rupezzz@mail.uni-paderborn.de
Paderborn University
Paderborn, Germany

Dominik Köhler
dominik.koehler@uni-paderborn.de
Paderborn University
Paderborn, Germany

Stefan Heindorf
heindorf@uni-paderborn.de
Paderborn University
Paderborn, Germany

Abstract

As machine learning and deep learning become increasingly integrated into our daily lives, understanding how these technologies make decisions is crucial. To ensure transparency, accountability, and ethical adherence, these so-called "black-box" models should be accompanied by human-comprehensible explanations of their predictions. This clarity is essential for establishing trust in their real-world applications. Similarly, it is crucial to compare different types of explanations to evaluate and understand their effectiveness, interpretability, and generalization capabilities for informed selection in various applications. To this end, we propose a framework called EDGE to evaluate diverse knowledge graph explanations, assessing logical rule-based and subgraph-based explanations by various explainers in terms of prediction accuracy and fidelity to the Graph Neural Network (GNN) model. Our evaluations reveal that logical methods excel in explaining complex and structured data, while subgraph-based models exhibit higher fidelity to the

in areas such as natural language processing [38], image recognition [36], and graph classification [19]. However, the lack of transparency in their decision-making processes has resulted in them being labeled as "black box" models. These models need to be accompanied by human-understandable explanations of their predictions to ensure transparency, accountability, and ethical considerations to build trust for their application in real-world scenarios.

Graph neural networks (GNNs) have also gained significant attention in recent years for their ability to model complex graph structures. While they have achieved state-of-the-art results in tasks such as node classification [27], link prediction [47], and graph classification [48], they again fall into the categories of "black box" models, due to the lack of transparency in their decision-making process. To provide explainability to GNNs, various explainers such as GNNExplainer [43], SubgraphX [46], PGExplainer [28] and XGNN [44] have been proposed in the recent literature.

While a few works [1, 2, 45] have proposed explanation methods

Starting point: EDGE paper

Approach	Pred. Perf.				Expl. Perf.			
	A	P	R	F1	A	P	R	F1
AIFB Dataset								
CELOE	0.722	0.647	0.733	0.688	0.739	0.682	0.743	0.711
EvoLearner	0.650	0.546	**0.973**	0.699	0.667	0.567	**0.973**	0.716
PGExplainer	**0.861**	**0.797**	0.920	**0.849**	**0.900**	**0.851**	0.948	**0.894**
SubGraphX	0.800	0.746	0.853	0.784	0.839	0.797	0.882	0.829

Code is available:

https://github.com/ds-jrg/EDGE

FAQ

Should I use strategy 1, 2, or 3?

- Only one approach is necessary (better have one high-quality approach than two medium-quality approaches)
- You can decide in your team which dataset, strategy, … you would like to use
- Strategy 1: Close to state-of-the art research (high predictive performance)
- Strategy 2: Closely related to lecture (maybe the easiest approach)
- Strategy 3: Close to state-of-the art research (global, logical explanation)
- If you are unsure, come and talk to me

Should the final project use only methods taught in the class?

- You can use methods taught in class
- But: you can use other methods, too

Is it ok to use a public repository for version control?

- Recommended to keep private for duration of course (to avoid plagiarism)

Can I use the example code in Panda?

- You can use it as a starting point
- But you should add your own contributions (see above)

Minimal expected content (for a typical mini project)

- **Introduction:** Definition of the machine learning task (what is predicted?)
- **Data analysis:** Table reporting dataset statistics
- **Model:** Table reporting model performance on training and test set
- **Explanation:** Figure showing an example explanation
  - Nodes and edges need to have human readable labels "Paul is a database professor"
  - It is not acceptable to just say: "node id 5 is connected to node id 7"
- **Evaluation:** Table reporting the quantitative evaluation of the explanations (e.g., automatic metrics or small user study)

- Your approach needs to be described in the text
- Each figure/table needs to be described in the text
  - How was it created?
  - What conclusions can you draw from it?

- **Own contribution.** At least one of:
  - More in-depth analysis of dataset
  - More in-depth analysis of GNN
  - Different explanation method
  - Different dataset